Architectural Style Justification

Problem Statement #43: Feature Flag & Dynamic Config Manager
Name: S Prithika
SRN: PES2UG24CS415

Architecture Selection: I chose Microservices Architecture for the Feature Flag & Dynamic Config Manager System.

Reason 1: Independent scaling of the hot path
Flag evaluation traffic is orders of magnitude higher than flag-management or config-override traffic, and must stay under the 15ms latency on the client's critical path. A microservices split lets the Flag Evaluation Service be scaled horizontally (and cached/replicated close to clients) independently of the lower-traffic management and configuration services, without over-provisioning the whole system as a single layered monolith would require.

Reason 2: Fault isolation for the kill-switch guarantee
A feature flag platform's core promise is that a flag can always be flipped off, even during an incident. Separating the Real-Time Sync Service and Audit & Logging Service from the Flag Evaluation Service means a slowdown or outage in sync or logging cannot block evaluation or overrides. In a Client-Server design, a single centralized server failing would take down evaluation, management, and the kill switch simultaneously.

Security Advantage:
Each service enforces its own scoped permission boundary. Role-based access control restricting production overrides to the Release Manager role is enforced inside the Environment Config Service, isolated from the Flag Evaluation Service's read-only evaluation path, so a bug in the high-traffic evaluation API cannot itself be used to change production flag state, and every config change is independently captured by the Audit & Logging Service as an immutable trail.

Performance Benefit:
The Flag Evaluation Service can be deployed as multiple lightweight, in-memory-cached instances placed close to client applications and scaled independently of write-heavy or I/O-heavy services (management, sync, audit). This keeps p99 evaluation latency under the required 15ms even as flag-management activity or audit volume grows, since those workloads run on entirely separate services and cannot contend for the evaluation service's resources.

UML Component Diagram

